

PM AI Bootcamp — Capstone Playbook: "Từ Đề tới Customer Zero" v2.0

Đọc khi nào: ngay sau khi nhóm nhận 1 trong 6 đề (PB-01...PB-06 trong *Project Brief Bank*) và bước vào **Phase 3 — Capstone "Build your Customer Zero"**. **Giải quyết nỗi lo:** "nhận đề xong rồi làm gì, theo thứ tự nào?" — tài liệu này là **đường ray từng bước** đi từ tờ đề → **một lát cắt đọc CHẠY ĐƯỢC**, đúng chuỗi mà RevenueOS (Customer Zero của FA) đã đi thật. **Nguyên tắc xuyên suốt:** *con người quyết định* — AI là *lực thực thi* (CASAN/Leash), **móng trước – bệ mặt sau, lát cắt đọc mỏng trước – bệ rộng sau**, mọi bước ghi **Dev Book + telemetry** ngay.

⚠ Quy tắc số 1 — AI gen ≠ đã xong

Playbook này khiến Claude Code sinh ra **rất nhiều artefact rất nhanh** — và đó chính là cái bẫy. **"Xong" không phải là "AI trả ra file"**, mà là **"bạn HIỂU và dám bảo vệ từng dòng"**. Vì vậy **mỗi bước** dưới đây có một 🛑 **Cổng hiểu**. Chưa qua cổng thì **coi như chưa xong bước đó — không đi tiếp**. Qua cổng cần 2 điều: 1. **Giải thích lại bằng lời của bạn** (không đọc lại file AI viết) — câu hỏi cụ thể nằm ở mỗi bước. 2. **Bắt được ≥1 chỗ AI làm sai/thiếu/thừa** và tự sửa — ghi lại. Không phản biện được điểm nào ⇒ khả năng cao bạn đang *rubber-stamp*.

Coach ở viva chấm **phán xử của bạn**, không chấm "AI trả lời hay". Nếu bỏ qua các 🛑, bạn sẽ có một chồng tài liệu đẹp mà **không hiểu gì** — đúng thứ chương trình này cấm.

o. Bản đồ 1 trang — 11 bước từ Đề tới Customer Zero

Đọc theo chiều mũi tên. **Mỗi bước sinh 1 artefact**, đặt tên gắn mã đề (ví dụ nhóm làm PB-01 → SPEC-PB01.md, WBS-PB01.md ...).

```
ĐỀ (PB-0X)
|
[0] Làm rõ scope ..... SCOPE-PB0X.md      ← chốt giả định, câu hỏi mở,
|
```

[1]	SW Spec chi tiết	SPEC-PB0X.md	← chức năng + NFR + use-case,
[2]	Module Map	MODULEMAP-PB0X.md	← chia module, MÓNG ẪN trước,
[3]	Architecture	ARCH-PB0X.md	← container + data model + API
[4]	WBS (Rolling-Wave) ...	WBS-PB0X.md	← bể task, bể sâu wave gần, c
[5]	Estimation	EST-PB0X.md	← ước lượng có telemetry
[6]	Risk + Governance	RISK-PB0X.md + DELEGATION-MAP-PB0X.md	← Least
[7]	Definition of Ready ..	DOR-PB0X.md	← chốt LÁT CẮT DỌC (Walking S
[8]	BUILD lát cắt dọc	code chạy + DEVBOOK-PB0X.md	← lái Claude Co
[9]	Test & Gate	SIT/UAT-PB0X	← cổng mặc-định-đóng, không
[10]	Trace + Telemetry	RTM-PB0X.md + telemetry + WEEKLY-PB0X.md	←
✓	VIVA	bảo vệ trước Coach – 3 cổng cứng: điểm · hệ thốn	

Bảng tra nhanh — bước ↔ artefact ↔ drill ↔ tham chiếu RevenueOS thật:

#	Bước	Artefact ra	Drill Workbook	RevenueOS đã làm (file tham chiếu)
0	Làm rõ scope	SCOPE-PB0X.md	—	PRD §0 <i>Quyết định nền (D1/D2)</i>
1	SW Spec chi tiết (SRS)	SPEC-PB0X.md	EX-01	PRD v0.2 + USE-CASE theo Role
2	Module Map & phân tầng	MODULEMAP-PB0X.md	—	MODULE MAP & BACKLOG (L0-L1)
3	Architecture	ARCH-PB0X.md	—	ARCHITECTURE.md + TECH-SPEC Phase 1
4	WBS + Rolling-Wave	WBS-PB0X.md	EX-02	Work Breakdown & Rolling-Wave Plan
5	Estimation	EST-PB0X.md	EX-03	(nằm trong Work Breakdown / backlog)
6	Risk + Delegation	RISK-PB0X.md , DELEGATION-MAP-PB0X.md	EX-04 + EX-06	Chiến lược Dev tự hành L4
7	Definition of Ready	DOR-PB0X.md	—	Build-Readiness & Definition of Ready (L4)
8	Build lát cắt dọc	code + DEVBOOK-PB0X.md	—	RevenueOS/ (apps/packages) + DEV_STATUS.md
9	Test & Gate	SIT/UAT records	(T6)	SIT/ UAT/ + CONFLICT-CHECK

#	Bước	Artefact ra	Drill Workbook	RevenueOS đã làm (file tham chiếu)
10	Trace + Telemetry	RTM-PB0X.md + telemetry	EX-05	RTM – Traceability + Analytics/

6 drill EX-01→EX-06 không biến mất — chúng là *bài tập rèn kỹ năng cho từng bước*. Playbook này là **sợi chỉ khâu chúng lại** thành một mạch chạy từ đề tới hệ thống, thêm 3 bước build/test/trace mà drill lẻ chưa phủ.

Cách lái Claude Code ở MỌI bước (vòng lặp 7 nhịp — nhắc lại)

Context → Plan → Delegate → Execute → Gate → Log → Iterate. Chi tiết: *Orchestrator Guide*. Ở mỗi bước dưới đây, phần "**Lái thế nào**" cho sẵn *prompt mẫu* — **bạn phán xử kết quả, không chấm "AI trả lời hay"**.

[o] Nhận đề & làm rõ scope

Mục tiêu: biến tờ đề (mơ hồ có chủ đích) thành một **phạm vi chốt được**: cái gì làm MVP, cái gì để sau, giả định nào đang gánh. **Input:** đề PB-0X + phần "*Câu hỏi làm rõ*" / "*WBS để sót*" trong brief. **Lái thế nào:**

"Đây là đề PB-0X (dán brief). Đóng vai PM: liệt kê **10 câu hỏi làm rõ** quan trọng nhất về scope/ràng buộc/dữ liệu; với mỗi câu, đề xuất **giả định mặc định** nếu chưa có câu trả lời. Đánh dấu 3 quyết định **ràng buộc kiến trúc** phải chốt trước khi thiết kế."

Output — SCOPE-PB0X.md : (1) 1 đoạn *problem statement*; (2) danh sách giả định đã chốt; (3) **quyết định nền** kiểu D1/D2 (ví dụ: MVP standalone hay tích hợp? input gì trước?); (4) *out-of-scope* rõ ràng. **Bẫy hay sót:** nhảy vào tính năng khi scope còn mơ hồ; không ghi giả định (sau này cãi nhau). **Done khi:** đọc SCOPE là biết MVP gồm gì, bỏ gì, và đang cược vào giả định nào. 🛡️ **Cổng hiểu:** nói bằng lời của bạn — *MVP này CÓ TÌNH bỏ cái gì, vì sao bỏ được?* + chỉ ra **≥1 giả định** AI tự đề xuất mà bạn không đồng ý (hoặc sửa lại cho đúng đề).

RevenueOS đã làm: PRD §0 chốt **D1 (MVP standalone, CRM/ERP để phase sau)** và **D2 (đa input Voice/Text/Ảnh/File cho MVP; Video/Bot phase 2)**. Nguyên tắc phân phase: *chạy trên dữ liệu nội bộ* → vào MVP; *cần tích hợp ngoài / media nặng* → hoãn.

[1] SW Spec chi tiết (SRS) — *drill EX-01*

Mục tiêu: requirement đủ để thiết kế & kiểm thử: **chức năng + phi chức năng (NFR) + use-case theo vai + tiêu chí chấp nhận (AC)**. **Input:** SCOPE-PB0X.md . **Lái thế nào:**

"Từ scope này, viết **SRS** gồm: (a) danh sách **user story** theo mẫu *Là , tôi muốn , để* ; (b) mỗi story kèm **Acceptance Criteria** (Given/When/Then); (c) **NFR** (bảo mật, hiệu năng, phân quyền, khả dụng); (d) **use-case theo từng role**. Đánh dấu story nào thuộc MVP."

Output — SPEC-PB0X.md : user story + AC + NFR + bảng **use-case × role**. **Bẫy hay sót:** chỉ có "happy path", quên **negative case** & phân quyền theo role; NFR bị bỏ trống; AC không đo được. **Done khi:** mỗi story MVP có AC kiểm được; có bảng role × use-case; NFR ghi rõ con số/tiêu chí. 🗝️ **Cổng hiểu:** chọn 1 user story bất kỳ, đọc AC rồi tự nói *ca nào PASS, ca nào FAIL* + bắt **≥1 story** AI chỉ viết happy-path (thiếu negative-case hoặc phân quyền theo role) và bổ sung.

RevenueOS đã làm: PRD v0.2 (14 module, user story) + tài liệu riêng **USE-CASE theo Role** — 7 role (sale/manager/bu_head/finance/bod/hr_head/admin), mỗi role có vùng use-case + **negative test** + cột "kịch bản demo/test". Bài học: *phân quyền là requirement, không phải chuyện kỹ thuật để sau* (vd role hr_head mù margin).

[2] Module Map & phân tầng — *móng ẩn trước, bề mặt sau*

Mục tiêu: chia hệ thống thành **module**, tách **móng-ẩn (Layer 0: auth, data model, phân quyền, gateway)** khỏi **module bề mặt**; chốt **scope MVP** (làm gì trước). **Input:** SPEC-PB0X.md . **Lái thế nào:**

"Nhóm các story thành **module**. Tách rõ **Layer 0 (nền tảng dùng chung)** vs **module bề mặt (người dùng thấy)**. Đề xuất **thứ tự làm**: móng nào phải xong trước để bề mặt chạy được. Chỉ ra 1 lát cắt dọc mỏng đi xuyên từ móng lên 1 tính năng bề mặt."

Output — MODULEMAP-PB0X.md : bảng module (Layer 0 / bề mặt) + scope MVP + ứng viên lát cắt dọc. **Bẫy hay sót:** làm bề mặt đẹp trên nền chưa có (auth/phân quyền/data model) → xây lại; ôm hết module cùng lúc. **Done khi:** thấy rõ "móng gì làm trước" và một lát cắt dọc mỏng để build đầu tiên. 🗝️ **Cổng hiểu:** chỉ vào sơ đồ và nói *móng nào PHẢI xong trước, vì sao bề mặt không chạy nếu thiếu nó* + bác **≥1 module** AI xếp sai tầng (coi bề mặt là móng hoặc ngược lại).

RevenueOS đã làm: MODULE MAP & BACKLOG (L0-L1) — QĐ-A7: MVP = Layer 0 đầy đủ + 6 module bề mặt + vài "móng-ảnh" của module sau; M3 Proposal Factory lùi P1.5. Có hẳn mục "Walking Skeleton — lát cắt đọc mỏng".

[3] Architecture

Mục tiêu: bản thiết kế kỹ thuật đủ để bắt đầu code: **sơ đồ container** · **data model (ERD)** · **API contract** · **tech stack** · **mô hình phân quyền**. **Input:** SPEC-PB0X.md + MODULEMAP-PB0X.md . **Lái thể nào:**

"Thiết kế kiến trúc cho MVP: (1) **nguyên tắc kiến trúc** (3–5 gạch đầu dòng); (2) **sơ đồ container** (web/mobile/API/DB/dịch vụ ngoài); (3) **data model** — bảng + trường + **độ nhạy dữ liệu**; (4) **API contract** cho các endpoint chính; (5) **tech stack** đề xuất kèm lý do; (6) **mô hình phân quyền** map tới use-case×role ở bước 1. Nêu rõ phần nào *chưa chốt*."

Output — ARCH-PB0X.md : đủ 6 mục trên; đánh dấu quyết định "chưa chốt". **Bẫy hay sót:** chọn stack theo "quen tay" không theo ràng buộc đề (vd PB-06 hợp đồng: *được gửi ra API ngoài hay bắt buộc self-host?* — quyết định này ràng buộc cả stack lẫn rủi ro); bỏ quên phân quyền ở tầng dữ liệu. **Done khi:** một Dev đọc ARCH là bắt tay dựng được; data model & API khớp story MVP. **Cổng hiểu:** nói được *vì sao chọn stack/kiểu phân quyền này thay vì cách khác* (đánh đổi gì) + tìm **≥1 trường dữ liệu** AI quên gắn độ nhạy và tự gắn.

RevenueOS đã làm: ARCHITECTURE.md (nguyên tắc · container · **phân quyền 5 lớp PEP-PDP-RLS** · AI pre-retrieval governance · **ERD** · capture pipeline · tech stack "đề xuất, chưa chốt") + TECH-SPEC Phase 1 (data model bổ sung + **API contracts** + thuật toán + UI flow theo màn hình). Bài học: **độ nhạy dữ liệu** gắn ngay vào từng trường, không dán sau.

[4] WBS + Rolling-Wave — *drill EX-02*

Mục tiêu: bẻ việc thành **đơn vị vừa làm được**, theo **rolling-wave**: wave gần bề **sâu (task)**, wave xa bề **thô (feature/epic)** — cố ý, để không "lập kế hoạch giả" cho thứ chưa rõ. **Input:** ARCH-PB0X.md + MODULEMAP-PB0X.md . **Lái thể nào:**

"Bẻ WBS cho MVP theo **rolling-wave**: Wave 1 (làm ngay) bề tới **task ~0.5–2 ngày**, Wave 2 bề mức **feature**, Wave 3+ bề **epic**. Mỗi task Wave 1 có: mô tả, output, phụ

thuộc. **Rà các bước hay bị sót** theo gợi ý trong brief."

Output — WBS-PB0X.md : cây việc phân wave, grain giảm dần theo độ xa. **Bẫy hay sót (xem brief từng đề!)**: quên tích hợp/đối soát (PB-01 cổng thanh toán, PB-04 làm sạch dữ liệu), quên màn quản trị/phân quyền/log kiểm toán, quên **kiểm thử** (sandbox, thiết bị thật, OCR/scan). **Done khi**: Wave 1 gồm task chạy được ngay; các "sót kinh điển" của đề đã có mặt. **Cổng hiểu**: cảm 1 task Wave 1 giải thích *xong nó ra cái gì, phụ thuộc ai* + bắt **≥1 việc hay sót** (tích hợp/đối soát/kiểm thử/màn quản trị/log) mà AI chưa đưa vào WBS.

RevenueOS đã làm: Work Breakdown & Rolling-Wave Plan — có rubric "một đơn vị việc vừa đúng"; Wave 1 grain **TASK**, Wave 2 grain **FEATURE** (coarser — cố ý), Wave 3–4 chỉ **EPIC**. Backlog quản trong Wave 0/1/2 Task Backlog.xlsx .

[5] Estimation — *drill EX-03*

Mục tiêu: ước lượng công sức có cơ sở, đo bằng **giờ người thật** (không suy từ token).

Input: WBS-PB0X.md . **Lái thế nào**:

"Ước lượng effort cho task Wave 1 (đơn vị **giờ/MD**), nêu **giả định & khoảng tin cậy** (lạc quan/khả dĩ/bi quan). Chỉ ra 3 task **rủi ro ước lượng cao nhất**."

Output — EST-PB0X.md : bảng estimate + giả định + khoảng. **Bẫy hay sót**: để AI phun một con số không giả định; quên buffer cho tích hợp/kiểm thử; tin móc đề đưa mà không kiểm. **Done khi**: mỗi estimate gắn giả định; tổng khớp móc đề hoặc chỉ rõ lệch ở đâu. **Cổng hiểu**: chỉ 1 estimate và nói *giả định nào đứng sau con số đó* + bác **≥1 con số** AI phun không giả định, buộc nó nêu khoảng lạc quan/khả dĩ/bi quan.

[6] Risk + Governance (Delegation Map) — *drill EX-04 + EX-06*

Mục tiêu: rủi ro có **người sở hữu + giảm thiểu**, và **quy tắc ủy quyền cho AI** (được tự làm gì / phải chờ duyệt gì). **Input**: WBS-PB0X.md + RISK sơ bộ. **Lái thế nào**:

"① **Lập Risk List**: rủi ro · xác suất · tác động · **mitigation** · owner — nhớ nhóm rủi ro **AI-sinh** (hallucination, lộ dữ liệu vào prompt, rubber-stamping). ② **Lập Delegation Map**: mỗi loại việc gắn mức **L0–L5** và **Leash A** (tới bản nháp) hay **A+** (việc rủi ro → cổng **fail-closed**, chờ người duyệt)."

Output: RISK-PB0X.md + DELEGATION-MAP-PB0X.md . **Bẫy hay sót:** quên rủi ro-AI; đặt Leash A+ cho việc đựng bảo mật/dữ liệu khách rồi vẫn "cho qua"; không ai sở hữu rủi ro.
Done khi: mỗi việc đựng dữ liệu nhạy/bảo mật đều ở **A+ + cổng fail-closed**; rủi ro-AI có mặt.
Cổng hiểu: giải thích vì sao 1 việc cụ thể phải để A+ chứ không A (chuyện gì xảy ra nếu A) + bắt **≥1** việc đựng dữ liệu nhạy mà AI xếp nhầm xuống Leash A.

RevenueOS đã làm: Chiến lược Dev tự hành L4 + 3 cơ chế governance: **fail-closed gate** · **BUILT-flagged** chờ duyệt · **hard-stop** khi mâu thuẫn. Leash A/A+ = 2 nấc cắt từ thang L0–L5 (A≈L3, A+≈L4).

[7] Definition of Ready — chốt lát cắt dọc để BUILD

Mục tiêu: trước khi code, xác nhận đủ điều kiện vào build cho lát cắt dọc đầu tiên (Walking Skeleton) — không phải "một bar cho tất cả", mà phân tầng theo mức tự hành.
Input: tất cả artefact [1]–[6]. **Lái thế nào:**

"Chốt lát cắt dọc đầu tiên: đi xuyên từ móng (auth/data) → 1 API → 1 màn hình chạy được. Viết **Definition of Ready** cho nó: spec rõ? data model có? API contract có? AC đo được? mức Leash? cổng kiểm là gì? Nếu thiếu mục nào → **chưa được build**."

Output — DOR-PB0X.md : checklist DoR **PASS/FAIL** cho lát cắt đầu + mô tả Walking Skeleton. **Bẫy hay sót:** build khi spec/data/AC còn trống → sửa đi sửa lại; chọn lát cắt quá dày. **Done khi:** DoR của lát cắt đầu **PASS** toàn bộ mục; lát cắt đủ mỏng để xong trong vài ngày. **Cổng hiểu:** vẽ/nói lát cắt dọc của bạn đi xuyên những tầng nào (móng → API → màn hình) + chỉ **≥1** mục DoR còn FAIL và vì sao chưa được build khi thiếu nó.

RevenueOS đã làm: Build-Readiness & Definition of Ready (L4) — DoR phân tầng theo autonomy của Epic; có mục "Áp vào Walking Skeleton — lát L4 đầu tiên làm gì"; DoR = cổng vào harness (chưa PASS thì không giao cho AI chạy tự hành).

[8] BUILD lát cắt dọc (Customer Zero) — tự tay lái AI dựng thứ CHẠY ĐƯỢC

Mục tiêu: một luồng chạy thật xuyên hệ thống (không phải slide/tài liệu). Đây là bằng chứng tốt nghiệp số 1. **Input:** DOR-PB0X.md PASS. **Lái thế nào** (vòng 7 nhịp, lặp từng task):

"Ta build lát cắt <tên> . Bấm ARCH + SPEC . **Trình plan từng bước rồi chờ tôi duyệt** (Plan mode). Được sửa code/chạy lệnh trong phạm vi Leash A; việc đụng <schema/secret/...> là **A+** → **dừng hỏi tôi**. Sau mỗi bước: chạy test, báo kết quả."

Ghi song song — DEVB00K-PB0X.md (bắt buộc): mỗi lần **AI sai** → **bạn sửa** ghi lại: AI làm gì, sai chỗ nào, bạn sửa ra sao, mức L & lý do, cổng nào chặn, có hard-stop không. *Đây là bằng chứng bạn HIỂU, không rubber-stamp.* **Bắt hay sót:** để AI chạy tự do không plan/gate; duyệt bừa output ("nhìn có vẻ đúng"); không ghi Dev Book ngay → tối viết lại = bịa. **Done khi:** demo được luồng chạy end-to-end; Dev Book có các điểm AI-sai/PM-sửa thật. 🗝️ **Cổng hiểu (ng nghiêm nhất):** sau mỗi bước AI làm, nói lại bằng lời *nó vừa đổi gì, vì sao đúng + bắt buộc ghi ≥1 điểm AI-sai* → **bạn-sửa THẬT** vào Dev Book. Dev Book trống trơn = bạn đang rubber-stamp = **chưa đạt bước này**.

RevenueOS đã làm: code thật ở Revenue05/ (apps/packages/policy/tests), tiến độ ghi ở DEV_STATUS.md ; kỷ luật *log data + rebuild dashboard cùng nhịp với việc làm* (definition-of-done của mọi task).

Hai nấc Capstone — chọn theo nền tảng kỹ thuật

Mục tiêu chương trình là PM **điều phối AI delivery**, không phải biến PM thành developer. Cả 2 tier đều yêu cầu **hiểu hệ thống end-to-end**, **ghi Dev Book thật**, và **demo được luồng chạy**. Khác nhau ở độ sâu kỹ thuật.

	Tier 1 — Full-stack	Tier 2 — PM-stack
AI chọn	PM có nền kỹ thuật hoặc đã từng code	PM thuần quản lý, non-coder
Frontend	Code chạy thật (Next.js / React)	Prototype tương tác trên browser (HTML/CSS/JS hoặc Next.js static) — bấm nút → chuyển trang → hiển thị dữ liệu
Backend	API chạy thật (NestJS / Spring Boot)	API mock (json-server hoặc mock endpoint). PM định nghĩa đúng API contract (endpoint, input, output, error code)
Database	PostgreSQL chạy thật + migration	Schema SQL viết đúng (PM lái AI sinh, review cấu trúc bảng, kiểm ràng buộc khoá ngoại, gán trường nhạy cảm) — không cần chạy migration thật
Test	Unit + integration test chạy xanh	PM viết test scenario bằng text (Given/When/Then), AI sinh test script, PM chạy và đọc kết quả pass/fail — không yêu cầu debug test code
Dev Book	Tập trung chỗ AI sinh code sai (logic, security, performance)	Tập trung chỗ AI sai ở tầng thiết kế (API contract sai field, schema thiếu trường, luồng UI không khớp AC)

	Tier 1 — Full-stack	Tier 2 — PM-stack
Demo	Luồng end-to-end chạy thật trên browser + DB	Luồng end-to-end chạy trên browser (prototype + mock data)

Chọn tier: học viên tự chọn + Coach xác nhận ở đầu Phase 3. **Không phạt chọn Tier 2** — cả 2 tier đều đủ điều kiện **Certified**. Tier 1 có lợi thế khi xét Distinction/Champion (bằng chứng sâu hơn) nhưng Tier 2 **không bị chặn tốt nghiệp**.

Vì sao có Tier 2: PM cần hiểu data model, API contract, test scenario — đó là kiến thức kiến trúc thiết yếu. Nhưng bắt PM debug runtime error NestJS → hoặc bỏ cuộc, hoặc rubber-stamp code vì không hiểu — đi ngược triết lý chương trình. Tier 2 giữ đúng phần PM **cần hiểu**, bỏ phần PM **không cần làm**.

[9] Test & Gate — cổng mặc-định-đóng

Mục tiêu: chứng minh lát cắt **đúng**, không "cảm giác đúng". **Không xác minh được = chặn**, không tạm cho qua. **Input:** code từ [8] + AC từ SPEC + negative test theo role. **Lái thế nào:**

"Sinh **test** cho lát cắt theo AC (happy + **negative** + phân quyền theo role). Chạy, báo pass/fail. Với ca fail: chỉ nguyên nhân, đề xuất sửa — **tôi duyệt** rồi mới sửa."

Output: kết quả SIT/UAT (đối chiếu AC), danh sách defect + trạng thái. **Bẫy hay sót:** chỉ test happy path; bỏ negative/phân quyền; "gần đúng nên cho qua". **Done khi:** AC của lát cắt pass; negative & phân quyền có test; defect còn lại được ghi nhận, không giấu. 🗝️ **Cổng hiểu:** đọc 1 ca test và nói *nó đang kiểm điều gì trong AC* + bắt **≥1 ca "gần đúng"** AI định cho qua và giữ cổng **đóng** (fail-closed) thay vì tắc lưới.

RevenueOS đã làm: thư mục SIT/ UAT/ + CONFLICT-CHECK W1 — Req-SIT-UAT-Trace (soát mâu thuẫn giữa Requirement↔SIT↔UAT). Governance: **fail-closed**.

[10] Traceability + Telemetry + Weekly Report — *drill EX-05*

Mục tiêu: (a) **truy vết** story → code → test (không rơi requirement); (b) **đo** bằng dữ liệu; (c) báo cáo ra quyết định được. **Input:** toàn bộ artefact + telemetry thu trong lúc build. **Lái thế nào:**

"① Lập **RTM**: mỗi user story MVP map tới artefact/task/test — chỉ ra story nào **chưa có test/chưa build**. ② Từ telemetry (giờ thật, số lần tôi sửa AI, token ước tính), dựng **mini-dashboard** + tính **Nén** (giờ truyền thống ÷ giờ thật) và **hiệu quả token**. ③ Viết **Weekly Report**: tiến độ · rủi ro · quyết định cần."

Output: RTM-PB0X.md (không còn story "mô côi") + bảng telemetry + WEEKLY-PB0X.md. **Bẫy hay sót:** để AI **bịa số** telemetry; đo token thay vì giờ thật; RTM bỏ sót story. **Done khi:** RTM phủ hết story MVP; số liệu là **giờ người thật** (est→reconcile), không proxy token. 🗝️ **Cổng hiểu:** chỉ 1 dòng RTM và truy *story này* → *code nào* → *test nào* + bác **≥1 số telemetry** nếu nghi AI bịa, xác nhận đó là giờ người thật chứ không suy từ token.

RevenueOS đã làm: RTM – Traceability & Coverage + checker `rtm_check.py` (PASS/GAP) chạy tự động khi sửa tài liệu; telemetry ở **Analytics/** (KPI **Nén** + **MD/1M-token**), đo **giờ ngồi máy thật**, token ghi *est* → *reconcile*.

✓ Chuẩn bị VIVA — 3 cổng cứng (thiếu 1 = trượt)

1. **Điểm 6 trụ năng lực** đạt ngưỡng (không trụ nào = 0).
2. **Hệ thống chạy được** — demo trực tiếp lát cắt dọc.
3. **Qua viva** — trả lời câu hỏi khó: *vì sao chọn kiến trúc này? chỗ nào AI sai và bạn sửa thế nào? nếu đổi ràng buộc X thì thiết kế đổi ra sao?*

Câu hỏi viva theo tier: - **Tier 1:** "Vì sao code xử lý edge case X kiểu này? Chỗ nào AI sinh code sai và bạn debug thế nào? Nếu đổi DB schema thì ảnh hưởng API nào?" - **Tier 2:** "Vì sao API này cần field X? Nếu thêm role mới thì schema đổi chỗ nào? Chỗ nào AI sinh API contract sai và bạn sửa thế nào?" — kiểm **tư duy hệ thống**, không kiểm kỹ năng code.

Bộ hồ sơ mang vào viva: SPEC · ARCH · WBS · RISK · DELEGATION-MAP · DOR · DEVBOOK · RTM · telemetry · WEEKLY + hệ thống chạy (Tier 1: code + test chạy; Tier 2: prototype + API contract + schema + test scenario). Coach chấm **phán xử của bạn**, không chấm "AI trả lời hay".

Coach hỏi gì? Rút thẳng từ 🗝️ **Cổng hiểu** của từng bước: chỉ 1 dòng bất kỳ trong artefact và hỏi "vì sao?", hoặc "chỗ nào AI sai và bạn sửa thế nào?". Nhóm **giải thích được** + có **Dev Book ghi lỗi AI thật** thì đã qua các cổng dọc đường; nhóm chỉ có chồng file đẹp sẽ lộ ngay ở câu hỏi đầu.

Phụ lục — Glossary

Tra cứu thuật ngữ: xem `docs/GLOSSARY.md` — bảng thuật ngữ thống nhất cho toàn bộ chương trình, có cột mapping sang thuật ngữ ngành quốc tế.

Tài liệu liên quan: *Project Briefs* (6 đề) · *Workbook Common* (EX-01→EX-06) + *Workbook nghề BA/SA/Dev* · *Orchestrator Guide* · *Program §7–§10* (assignment · rubric · KPI) · *Operating Model* (L0–L5 · Leash · governance).

Phụ lục — Changelog

Version	Ngày	Nội dung
v2.0	2026-08-24	Thêm 2-tier Capstone: bước [8] chia thành Tier 1 (full-stack, code chạy thật) và Tier 2 (PM-stack: prototype + API mock + schema + test scenario) cho PM non-coder. Viva cập nhật câu hỏi theo tier. Glossary thay bằng pointer tới <code>docs/GLOSSARY.md</code> . Changelog dời xuống cuối.
v1.1	2026-07-09	Thêm cơ chế chống rubber-stamp : hộp <i>"Quy tắc số 1 — AI gen ≠ đã xong"</i> đầu file +  Cổng hiểu ở cả 11 bước (mỗi cổng buộc <i>giải thích lại bằng lời + bất ≥1 lỗi AI</i>). Vá rùi ro "gen ào ào nhưng không hiểu gì".
v1.0	2026-07-09	Bản đầu: pipeline 11 bước từ nhận đề → hệ thống chạy → viva; mỗi bước có mục tiêu · input · cách lái Claude Code · output · bẫy hay sót · tiêu chí Done, kèm hộp "RevenueOS đã làm gì" tham chiếu artefact thật (PRD · Use-Case · Module Map · Architecture · Tech-Spec · Work Breakdown · DoR L4 · RTM · SIT/UAT · Telemetry). Ảnh xạ về 6 drill Workbook EX-01→EX-06.